



OOP

Tratamento de Erros

Prof. Edson José Pacheco

2026

Fundamentos e Filosofia

- ▶ Comparar códigos de retorno com fluxo de exceções.
- ▶ Aplicar o princípio **EAFP** para códigos robustos.
- ▶ Diferenciar falhas de hardware e erros de lógica.

Modelo Orientado a Objetos

- ▶ Analisar a hierarquia da classe `BaseException`.
- ▶ Implementar tratamento via **Polimorfismo**.
- ▶ Manipular objetos de exceção e seus metadados.

Implementação Avançada

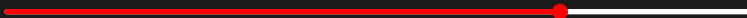
- ▶ Criar **Classes de Exceção** customizadas.
- ▶ Utilizar **Exception Chaining** (`raise from`).
- ▶ Gerir recursos via blocos `finally` e `with`.

Diagnóstico e Estabilidade

- ▶ Validar lógica interna com o comando `assert`.
- ▶ Integrar logging e traceback profissional.
- ▶ Escolher ferramentas via árvore de decisão técnica.



CLIQUE PARA ABRIR NO NAVEGADOR



 Um *Overview* sobre Exceptions

Historicamente, em linguagens como C e Assembly, o tratamento de erros baseia-se na verificação manual de **códigos de retorno**.

O Padrão Clássico (C/Python):

Python

```
res = sensor.inicializar()
if res == 0: # Sucesso
    res = sensor.ler()
    if res != -1: # Erro de leitura?
        processar(res)
    else:
        print("Erro na leitura")
else:
    print("Erro na inicialização")
```

Limitações e Problemas:

- ▶ **Código Poluído:** A lógica principal fica "escondida" entre dezenas de verificações *if/else*.
- ▶ **Falhas Silenciosas:** É fácil esquecer de verificar um retorno, levando a comportamentos imprevisíveis.
- ▶ **Propagação:** O erro deve ser passado manualmente de função em função.

Existem duas formas principais de lidar com erros em programação. O Python abraça o modelo baseado em exceções.

- ▶ **LBYL (*Look Before You Leap*)**: Verificar todas as condições antes de agir (*if/else* massivo). Comum em C/C++.
- ▶ **EAFP (*Easier to Ask for Forgiveness than Permission*)**: "É mais fácil pedir perdão do que permissão". Tenta-se a operação e trata-se a falha se ela ocorrer.

Vantagem em POO

O modelo **EAFP** produz código mais legível e rápido em situações comuns, pois o "custo" da exceção só é pago se o erro de fato ocorrer.

No hardware, eventos inesperados (como uma interrupção de erro no barramento I2C) exigem uma resposta imediata que o fluxo sequencial do *if/else* nem sempre consegue atender.

- ▶ **Eventos de Hardware:** Falhas físicas (ex: desconexão de um sensor SPI) geram sinais que precisam "interromper" a lógica comum.
- ▶ **Motivação Moderna:** As linguagens modernas (C++, Python, Java) criaram as exceções para separar o **fluxo feliz** (execução normal) do **fluxo de exceção** (tratamento de erros).
- ▶ **Desacoplamento:** O programador foca no que o sistema deve fazer, e centraliza o que ele deve fazer "se algo der errado".

O bloco **try** substitui as verificações aninhadas por uma área de execução monitorada. Se qualquer instrução falhar, o controle é desviado imediatamente.

Antes (Aninhamento)

Python

```
if inicializar():  
    if conectar():  
        if ler():  
            salvar()  
        else: erro()  
    else: erro()  
else: erro()
```

Depois (Bloco Protegido)

Python

```
try:  
    inicializar()  
    conectar()  
    ler()  
    salvar()  
except:  
    tratar\_qualquer\_erro()
```

Vantagem Estrutural

O código torna-se linear e legível. A detecção do erro é automática: você não precisa mais perguntar ao hardware "deu erro?" a cada linha; o próprio sistema "lança" o erro para você.

Em sistemas avançados, falhas são inevitáveis (erro de rede, sensor desconectado, estouro de buffer). O tratamento de exceções permite que o sistema **recupere o controle** em vez de travar.

Sem Tratamento

- ▶ O programa é interrompido abruptamente.
- ▶ O hardware pode ficar em estado inconsistente.
- ▶ Perda total de dados de telemetria.

Com Tratamento

- ▶ Erros são capturados e registrados (Log).
- ▶ O sistema pode tentar uma reconexão.
- ▶ Garante o encerramento seguro de recursos.

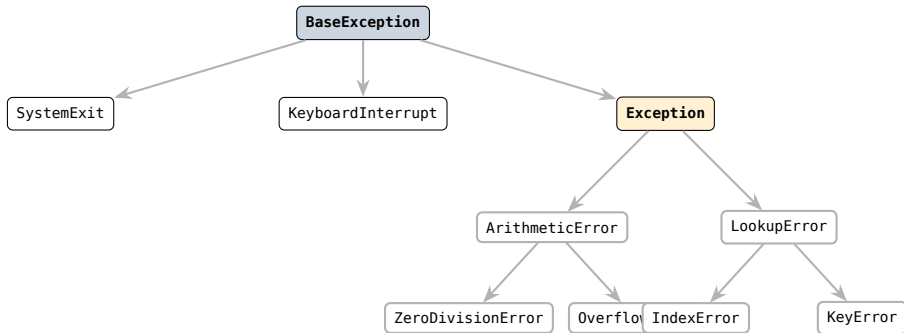
No Python, o tratamento de erros é integralmente baseado em objetos. Uma exceção não é um simples sinalizador, mas uma **instância de classe**.

- ▶ **Atributos:** Exceções podem carregar metadados do erro (mensagens, códigos de estado, logs de sensores).
- ▶ **Métodos:** Por serem classes, podem implementar lógica de recuperação ou formatação personalizada.
- ▶ **Ciclo de Vida:** Quando ocorre um erro, o interpretador *instancia* o objeto da exceção e o "lança" no fluxo do programa.

Conceito Técnico

A sintaxe `except ValueError as e:` cria uma referência local `e` que aponta para o objeto instanciado, permitindo acessar suas propriedades via introspecção.

A eficácia do tratamento reside na compreensão da árvore de herança. O Python utiliza herança múltipla e simples para categorizar erros.



Princípio do Polimorfismo: Capturar uma classe base (ex: `ArithmeticError`) intercepta automaticamente todas as suas subclasses (ex: `ZeroDivisionError`).

Por ser uma instância de classe, o objeto da exceção carrega informações valiosas para o diagnóstico do sistema.

Python

```
try:
    1 / 0
except ZeroDivisionError as e:
    print(f"Tipo: {type(e)}")      # Classe do erro
    print(f"Args: {e.args}")      # Argumentos da instanciação
    print(f"Mensagem: {str(e)}")  # Representação textual
```

- ▶ **args:** Uma tupla contendo os argumentos passados ao construtor.
- ▶ **`__traceback__`:** Um objeto que contém a pilha de chamadas ligada a essa instância específica.

Devido à herança, a ordem dos blocos `except` define a precisão do tratamento. O Python avalia os blocos de cima para baixo.

Python

```
try:
    sensor.configurar(limite=500)
    valor = lista_amostras[10]
except IndexError:
    # Captura a classe específica (Filha)
    print("Amostra não encontrada")
except Exception as e:
    # Captura a classe genérica (Pai)
    print(f"Erro inesperado: {type(e).__name__}")
```

Regra de Ouro:

- ▶ Sempre trate as exceções **mais específicas** primeiro.
- ▶ O uso de `except Exception` deve ser o último recurso para logs genéricos.

Para sistemas robustos, é prática recomendada criar classes de erro que reflitam o domínio da aplicação, herdando sempre de **Exception**.

Python

```
class FalhaConexaoError(Exception):
    """Exceção para erros de hardware I2C/SPI."""
    def __init__(self, porta, tentativa):
        self.porta = porta
        self.tentativa = tentativa
        super().__init__(f"Falha na porta {porta} "
                        f"após {tentativa} buscas.")

# Disparando a exceção
raise FalhaConexaoError("UART1", 3)
```

Por que customizar?

- ▶ Melhora a **legibilidade** técnica.
- ▶ Permite passar parâmetros contextuais para o log.
- ▶ Facilita a filtragem em sistemas multicamadas.

O comando **raise** é utilizado para interromper o fluxo normal do programa e sinalizar explicitamente que uma condição anômala ocorreu.

Instanciação e Disparo:

Python

```
def validar_idade(idade):  
    if idade < 0:  
        # Instancia e lança o objeto  
        raise ValueError("Idade negativa")  
    return True  
  
# O interpretador para aqui e  
# busca um tratador (handler)  
validar_idade(-5)
```

Funcionamento OO:

- ▶ **Gatilho:** Transfere o controle do "Fluxo Feliz" para o "Fluxo de Exceção".
- ▶ **Objeto Vivo:** O raise exige uma instância de classe derivada de BaseException.
- ▶ **Semântica:** Permite que o desenvolvedor defina suas próprias regras de negócio como erros formais.

A relação entre **raise** e **except** assemelha-se a um arremesso de bola: um ponto do código lança o objeto e outro ponto deve estar pronto para capturá-lo.



Re-raising (Repropagação):

- ▶ Usar `raise` sem argumentos dentro de um `except` lança novamente a mesma exceção.
- ▶ Útil para fazer log e permitir que o erro suba na pilha.

Propagação Automática:

- ▶ Se um `raise` não encontrar um `except` local, ele "sobe" para a função que a chamou.
- ▶ Se ninguém capturar, o programa encerra.

Em sistemas complexos, uma exceção de baixo nível pode ser a causa de um erro de alto nível. O Python permite preservar essa história.

Python

```
try:
    config = os.open("perfil.json")
except OSError as erro_origem:
    # Eleva o erro preservando a causa original
    raise ErroInicializacao("Falha no boot") from erro_origem
```

Rastreabilidade (Traceback)

O uso de `from` garante que o relatório de erro exiba a sequência lógica das falhas, facilitando o *debugging* em ambientes de produção.

Em situações onde diferentes falhas exigem a mesma estratégia de recuperação, podemos capturar múltiplas classes em um único bloco.

Sintaxe com Tuplas:

Python

```
try:
    # Operação de I/O e Conversão
    dado = int(input("Valor: "))
    processar(dado)
except (ValueError, TypeError):
    # Ambos os erros são tratados aqui
    print("Erro: Entrada inválida.")
except (IOError, EOFError):
    print("Erro: Falha na comunicação.")
```

Vantagens:

- ▶ **DRY (Don't Repeat Yourself):**
Evita a duplicação de código de tratamento.
- ▶ **Organização:** Agrupa erros por categorias lógicas (ex: erros de entrada vs. erros de sistema).

O bloco finally é o mecanismo que garante a execução de instruções de limpeza, independentemente da ocorrência de exceções.

Python

```
def ler_log_sistema():  
    arquivo = open("log.csv", "r")  
    try:  
        processar_dados(arquivo)  
    except ValueError:  
        print("Erro de formato")  
    finally:  
        # Executa SEMPRE (sucesso ou erro)  
        arquivo.close()  
        print("Arquivo encerrado com segurança")
```

Aplicações Críticas:

- ▶ Fechar conexões de rede.
- ▶ Liberar travas de hardware (Locks).
- ▶ Desligar periféricos de potência.
- ▶ Garantir 100% de desalocação.

O bloco `else` é executado apenas se o bloco `try` NÃO disparar nenhuma exceção. Ele serve para separar o código monitorado da lógica subsequente.

Uso Recomendado:

Python

```
try:
    f = open("config.json", "r")
except IOError:
    print("Falha ao abrir arquivo")
else:
    # Só executa se o arquivo abriu!
    config = f.read()
    processar_dados(config)
    f.close()
```

Por que usar?

- ▶ **Precisão:** Evita capturar acidentalmente uma exceção em `processar_dados()` que você não pretendia tratar.
- ▶ **Clareza:** O `try` fica restrito apenas à operação de risco (abrir o arquivo).

Diferente de uma exceção de runtime (como rede fora do ar), o **assert** dispara uma exceção (AssertionError) para indicar erros que **nunca** deveriam ocorrer.

Python

```
def calcular_area_circulo(raio):  
    # Verificação de invariante técnica  
    assert raio >= 0, "0 raio não pode ser negativo"  
    return 3.14159 * (raio ** 2)
```

- ▶ **Documentação Ativa:** Serve como garantia técnica para outros desenvolvedores.
- ▶ **Otimização:** Podem ser desativadas globalmente no Python para ganho de performance.

Em ambiente de produção, onde a performance é crítica, o Python permite ignorar todas as instruções `assert` sem a necessidade de alterar o código-fonte.

Comando via Terminal:

Execução Otimizada

```
python -O meu_script.py
```

O que acontece internamente?

- ▶ A variável global `__debug__` é definida como `False`.
- ▶ O bytecode para os `asserts` não é gerado.



Perigo Técnico

Nunca coloque lógica essencial dentro de um `assert`!

- ▶ **Errado:** `assert saldo.sacar(10)`
- ▶ **Correto:** `res = saldo.sacar(10)`
`assert res == True`

Se as asserções forem desativadas (`-O`), a lógica de "sacar" no exemplo errado **nunca será executada**.

Em sistemas orientados a objetos modernos, o padrão **with** substitui a estrutura try/finally para garantir o fechamento de recursos de forma automática.

A Abordagem OO Nativa:

Python

```
with open("dados.txt") as f:  
    conteudo = f.read()  
    # O arquivo fecha AQUI  
    # mesmo se houver erro!
```

Funcionamento Interno:

- ▶ Invoca o método `__enter__` no início.
- ▶ Garante a execução de `__exit__` ao final.
- ▶ É a forma mais segura de evitar vazamentos de memória (Leaks).

Nem toda anomalia deve interromper o sistema. O módulo **warnings** permite alertar o usuário sem desviar o fluxo de execução.

Python

```
import warnings

def configurar_sensor(ganho):
    if ganho > 100:
        warnings.warn("Ganho muito alto: risco de ruído",
                      RuntimeError)
    # O programa continua normalmente...
    executar_setup(ganho)
```

Diferença Técnica

Exceção: Erro fatal ou semântico que impede a lógica.

Warning: Alerta sobre obsolescência (*deprecation*) ou condições sub-otimizais.

Em sistemas embarcados e servidores, exceções devem ser registradas em arquivos de log persistentes, e não apenas impressas na tela.

Python

```
import logging
logging.basicConfig(filename='sistema.log', level=logging.ERROR)

try:
    resultado = operacao_de_risco()
except Exception as e:
    # Registra o erro e o Traceback automaticamente
    logging.exception("Falha crítica detectada")
```

- ▶ **Níveis:** DEBUG, INFO, WARNING, ERROR, CRITICAL.
- ▶ **Persistência:** Garante que o histórico de erros sobreviva a reinicializações (WDT).

O **Traceback** é o relatório gerado pelo Python que lista as chamadas de funções ativas no momento em que uma exceção não tratada ocorre.

- ▶ **Pilha de Chamadas:** Mostra a sequência "quem chamou quem" até o ponto da falha.
- ▶ **Leitura Inversa:** No Python, o erro mais recente é exibido por **último** no console.
- ▶ **Propagação:** Se uma função A chama B e ocorre um erro em B, o interpretador procura um except em B, depois em A, e assim por diante.

Exemplo de Saída no Console

```
Traceback (most recent call last):  
  File "main.py", line 10, in <module> → iniciar()  
  File "main.py", line 5, in iniciar → processar()  
  File "main.py", line 2, in processar → 10 / 0  
ZeroDivisionError: division by zero
```

Em certas situações, capturamos a exceção com `except`, mas ainda precisamos registrar o rastro completo para depuração. Para isso, utilizamos o módulo **traceback**.

Uso do `print_exc()`:

Python

```
import traceback

def operacao_complexa():
    raise RuntimeError("Falha no barramento")

try:
    operacao_complexa()
except Exception:
    print("Capturado! Imprimindo rastro:")
    traceback.print_exc()
```

Vantagens:

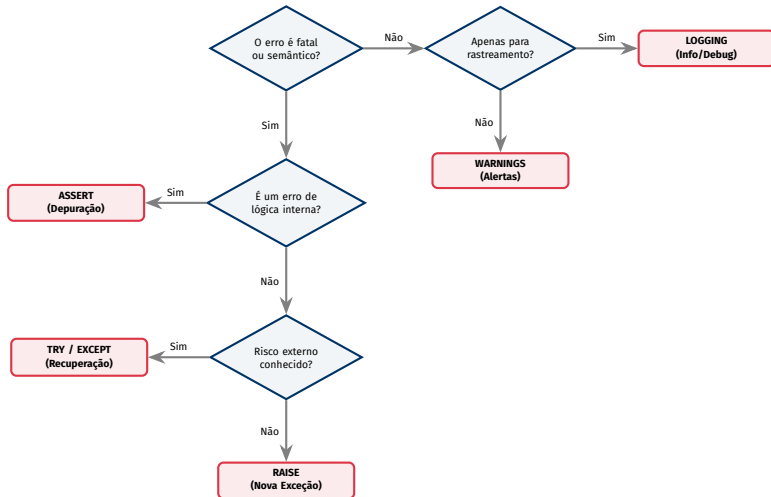
- ▶ **Depuração:** Permite ver o erro sem encerrar o programa.
- ▶ **Logs:** O rastro pode ser redirecionado para arquivos de texto.
- ▶ **Controle:** Ideal para sistemas que rodam ****24/7**** (como servidores ou IoT).

A escolha da estratégia correta depende da natureza da anomalia e do impacto no fluxo do sistema.

NOTAS TÉCNICAS

Resiliência: Use try/except para o que você não controla (sensores, rede, arquivos).

Maturidade: Substitua print por logging para garantir rastreabilidade em sistemas de produção.



Domine o Tratamento de Erros em Python: Filosofia EAFP e Robustez OO

Capacitar desenvolvedores a transitar do modelo clássico de códigos de retorno para o sistema moderno de exceções do Python, utilizando princípios de Orientação a Objetos

Filosofia e Estrutura de Controle



EAFP vs. LBYL

Python prioriza a EAFP (tentar a operação e tratar a falha) em vez de verificações manuais massivas.

EAFP

Princípio: Tente realizar a operação e trate a exceção se ela ocorrer.

O Fluxo de Exceção



O comando 'with' e o bloco 'finally' garantem o fechamento de arquivos e hardware, evitando vazamentos.

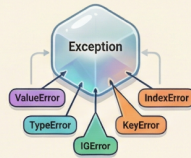
Gestão Segura de Recursos

Exceções são Objetos



Exceções são Objetos

Erros são instâncias de classes que carregam metadados, mensagens e o resto da pilha (traceback).



Polimorfismo na Captura

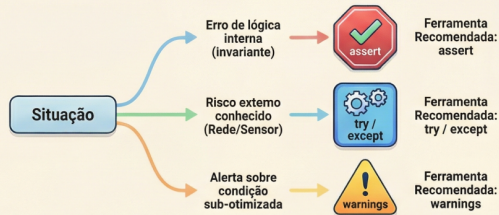
Capturar uma classe base (como Exception) intercepta automaticamente todas as suas subclasses específicas.



Exceções Customizadas

Criar classes próprias herdando de Exception melhora a legibilidade e a semântica do domínio técnico.

O Modelo Orientado a Objetos





Prof. Edson José Pacheco

pacheco@ufpr.br

***"Scientists study the world as it is; engineers create the world that has never been.
(Theodore von Kármán)"***